

TRABALHO AVALIATIVO

Redes Neurais Artificiais —MLP, CNN ou LSTM

Este documento apresenta possíveis soluções para os 10 problemas propostos no trabalho avaliativo de Redes Neurais Artificiais da aula do dia 13/05. Para cada problema, eu indico qual arquitetura mais adequada pra usar (MLP, CNN ou LSTM), explicando qual modelo que eu utilizaria e os motivos para não utilizaria os demais jeitos.

Henrique Cordeiro Da Silva

Problema 1: Diagnosticar pneumonia a partir de imagens de raio-X de tórax

Qual escolhi: CNN (Convolutional Neural Network)

Por que essa rede faz sentido aqui:

Imagens de raio-X são fotos normais. As marcas que indicam pneumonia (manchas, regiões esbranquiçadas no pulmão) aparecem em lugares específicos da imagem. A CNN foi feita exatamente pra isso: ela usa filtros que percorrem a imagem inteira procurando padrões locais, como bordas, texturas e formatos. O legal é que ela consegue reconhecer o mesmo padrão mesmo que ele apareça em cantos diferentes da imagem. As camadas mais fundas da rede vão juntando esses pedaços e formando uma ideia geral do que é uma região com pneumonia. No final, a rede usa uma função que diz "tem pneumonia" ou "não tem", já que são só duas opções possíveis.

Por que não usar outra rede:

MLP: ela joga fora a estrutura da imagem, transformando tudo num vetor sem sentido espacial. Seria como tentar reconhecer uma foto embaralhando todos os pixels — fica impossível.

LSTM: ela foi feita pra sequências no tempo, tipo frases ou dados de série. Pixels de uma foto parada não têm ordem no tempo, então usar LSTM aqui não faz sentido.

Problema 2: Prever o consumo de energia elétrica nas próximas 24 horas

Qual escolhi: LSTM (Long Short-Term Memory)

Por que essa rede faz sentido aqui:

O consumo de energia depende muito do que aconteceu antes: o horário de pico da manhã influencia o da tarde, o padrão de uma segunda-feira se repete nas seguintes etc. A LSTM é boa exatamente nisso: ela tem uma espécie de memória interna que guarda informações de horas e até dias atrás. Ela tem "portões" que decidem o que vale a pena lembrar e o que pode esquecer — isso ajuda muito quando o histórico é longo (tipo 7 dias, que são quase 170 horas de dados). No final, ela prevê um número (o consumo), então a saída é direta, sem função de escolha entre categorias

MLP: ela trataria cada hora como se fosse uma informação independente, sem ver que existe uma ordem no tempo. Se mudar o tamanho do histórico, teria que treinar tudo de novo.

CNN: ela consegue capturar alguns padrões locais em séries temporais (tipo um pico curto de consumo), mas tem dificuldade com dependências que se estendem por vários dias — o campo de visão dela é pequeno e fixo.

Problema 3: Classificar e-mails como spam ou não spam

Qual escolhi: MLP (Multilayer Perceptron)

Por que essa rede faz sentido aqui:

Quando o texto do e-mail é transformado em um vetor de palavras (tipo uma lista que diz quais palavras aparecem e quantas vezes), não existe uma ordem espacial ou temporal importante. O que define se um e-mail é spam é a presença de certas palavras, tipo "grátis", "promoção", "clique aqui" — e a MLP é boa em aprender essas combinações. Não importa se a palavra "oferta" aparece no começo ou no fim do e-mail: o que conta é que ela está lá. No final, a rede escolhe entre duas opções (spam ou não spam), então usa uma função de saída que dá um valor entre 0 e 1.

LSTM: ela leria o e-mail palavra por palavra na ordem do texto, mas pra detectar spam a ordem não importa muito. Além disso, e-mails longos causariam problemas no treinamento.

CNN: precisaria de embeddings e convoluções sobre o texto, o que complica tudo desnecessariamente quando a representação simples de palavras já resolve o problema.

Problema 4: Identificar o autor de um trecho de código-fonte pelo estilo de escrita

Qual escolhi: LSTM (Long Short-Term Memory)

Por que essa rede faz sentido aqui:

O jeito de cada programador escrever código aparece ao longo de toda a sequência: como ele nomeia variáveis, qual estrutura prefere usar, como organiza as funções. Essas características dependem da ordem — o padrão de como uma função é declarada lá no começo se conecta com a forma como é usada mais adiante. A LSTM consegue guardar essas informações ao longo da leitura do código, lembrando de coisas que vieram antes

pra fazer sentido das que vêm depois. A ordem dos tokens é essencial: o mesmo código embaralhado não teria o mesmo estilo. No final, a rede escolhe entre vários autores possíveis, então usa uma função de saída que dá a probabilidade de ser cada um

MLP: trataria o código como uma sacola de palavras sem ordem, perdendo completamente a estrutura sintática, e a lógica que é justamente o que identifica o estilo do programador.

CNN: captaria padrões locais curtos, mas teria dificuldade com o estilo que se manifesta ao longo de trechos maiores.

Problema 5: Detectar falhas mecânicas em motores industriais a partir do som

Qual escolhi: CNN (Convolutional Neural Network)

Por que essa rede faz sentido aqui:

O som do motor pode ser transformado em um espectrograma, transformando o som em imagens. Falhas mecânicas, como rolamentos danificados, criam padrões visuais característicos nesse espectrograma, faixas de frequência que se repetem. A CNN é ótima pra isso: ela detecta esses padrões locais na imagem e consegue reconhecê-los independentemente de onde aparecem no trecho de áudio. No final, a rede diz se tem falha ou não (ou qual tipo de falha).

Por que não usar outra rede:

MLP: jogaria fora a estrutura tempo-frequência do espectrograma, que é exatamente o que identifica as falhas.

LSTM: processar o áudio bruto como sequência seria muito pesado — o áudio tem dezenas de milhares de amostras por segundo, tornando as sequências longas demais.

Problema 6: Prever inadimplência bancária a partir de 30 atributos cadastrais

Qual escolhi: MLP (Multilayer Perceptron)

Os 30 dados (idade, renda, score de crédito, tempo como cliente, etc.) são informações variadas que não têm ordem espacial nem temporal. A MLP é ideal pra aprender relações complexas entre esses dados. Todos os dados são apresentados de uma vez pra rede, que

aprende as combinações importantes pelas camadas ocultas. No final, a rede decide entre duas opções (vai pagar ou não vai pagar)

Por que não usar outra rede:

CNN: tentaria encontrar padrões locais entre atributos no vetor de entrada, mas a ordem dos 30 atributos é arbitrária, não há localidade espacial real. Um filtro assim sobre [idade, renda, score] como se fossem pixels vizinhos não tem semântica.

LSTM: trataria os 30 atributos como uma sequência temporal, mas não existe uma ordem temporal entre eles

Problema 7: Reconhecer dígitos manuscritos (0 a 9) em formulários escaneados

Qual escolhi: CNN (Convolutional Neural Network)

Por que essa rede faz sentido aqui:

Dígitos escritos à mão são imagens com estrutura espacial clara: cada número é formado por traços, curvas e cruzamentos que se organizam no espaço. A CNN captura isso de forma hierárquica: as primeiras camadas detectam bordas e curvas simples; as do meio juntam esses elementos em partes do dígito; as finais reconhecem o número inteiro. Além disso, a CNN consegue reconhecer um "3" mesmo que ele esteja um pouco deslocado ou inclinado na imagem — isso é fundamental pra formulários reais. Usar filtros compartilhados também reduz muito a quantidade de parâmetros comparado a uma rede mais simples. No final, a rede escolhe entre 10 classes (0 a 9), usando uma função de saída que dá a probabilidade de ser cada dígito

Por que não usar outra rede:

MLP: acho que funciona. Mas um "7" deslocado dois pixels pra direita pareceria uma entrada completamente nova pra ela.

LSTM: tratar as linhas da imagem como se fossem passos no tempo é artificial e quebra a estrutura 2D do dígito — a rede veria cada linha sem contexto das colunas ao lado

Problema 8: Sugerir a próxima palavra provável em um aplicativo de chat

Qual escolhi: LSTM (Long Short-Term Memory)

Por que essa rede faz sentido aqui:

Essa daí já foi usada e mencionada em aula, prever a próxima palavra depende completamente da ordem das anteriores: "O gato comeu o..." leva a palavras bem diferentes de "O rato comeu o...". A LSTM guarda informações sobre o contexto da conversa inteira no estado oculto, então palavras ditas lá atrás ainda influenciam a predição atual. Os portões da rede aprendem a manter as informações importantes (tipo o assunto principal da frase) e descartar as que não importam mais. No final, a rede gera uma probabilidade pra cada palavra do vocabulário — a que tiver maior probabilidade é a sugerida.

Por que não usar outra rede:

MLP: sem memória de sequência, a MLP só conseguiria olhar para uma janela fixa de palavras anteriores. Perderia dependências de longo alcance — se o usuário mencionou o tema 10 palavras atrás, a MLP não conseguiria usar essa informação.

CNN: filtros convolucionais 1D mas têm campo receptivo limitado e fixo. Para predição de próxima palavra, o contexto pode ser arbitrariamente longo — a CNN não escala bem para capturar dependências de longa distância sem empilhar muitas camadas.

Problema 9: Classificar tumores como benignos ou malignos a partir de 10 medidas de biópsia

Qual escolhi: MLP (Multilayer Perceptron)

Por que essa rede faz sentido aqui:

As 10 medidas são números extraídos ao mesmo tempo, sem nenhuma temporal entre eles. O objetivo é aprender onde fica a fronteira que separa tumores benignos dos malignos nesse espaço de 10 dimensões. A MLP é perfeita pra isso: com poucas camadas ocultas ela aprende combinações entre as medidas — por exemplo, raio grande com baixa suavidade pode indicar malignidade. Com só 10 atributos, até métodos mais simples funcionariam bem;

Por que não usar outra rede:

CNN: aplicaria filtros convolucionais sobre os 10 atributos como se fossem uma sequência espacial, mas a ordem é arbitrária. Um filtro que olha para [raio, textura] localmente não tem fundamento físico, qualquer permutação dos atributos deveria dar o mesmo resultado, o que a CNN não garante.

LSTM: trataria as 10 medidas como uma sequência temporal onde a ordem importa, mas isso é falso, não há razão para que 'raio médio' preceda 'textura' em algum sentido causal ou temporal. A rede aprenderia dependências sequenciais fictícias.

Problema 10: Identificar a emoção predominante de um locutor a partir de áudio de 3 segundos

Qual escolhi: CNN (Convolutional Neural Network)

Por que essa rede faz sentido aqui:

Mesmo padrão pegar o trecho de áudio é transformar em um espectrograma). Emoções podem criar padrões acústicos diferentes como variações de tom, volume, energia em certas frequências e ritmo, que aparecem como texturas e formatos distintos nessa imagem. Dessa forma CNN detecta esses padrões locais no espectrograma: frequências próximas e janelas de tempo vizinhas contribuem juntas pro padrão emocional.. No final, a rede escolhe entre várias emoções possíveis, usando uma função que dá a probabilidade de cada uma.

Por que não usar outra rede:

LSTM: processar o áudio bruto como sequência seria muito pesado, o áudio tem dezenas de milhares de amostras por segundo, tornando as sequências longas demais. Talvez transcrevendo para áudio mas seria trabalhoso

MLP: jogaria fora a estrutura tempo-frequência do espectrograma, que é exatamente o que identifica emoções.